



Cloud Computing & Dev-Ops



Think Cloud. Act DevOps. Be Unstoppable

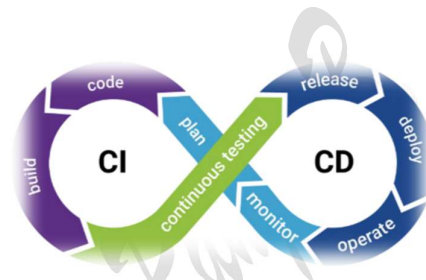
“GitHub Actions: CI/CD: Automating Your Development Workflow.”

DevOps at a Glance

DevOps is a combination of cultural philosophies, practices, and tools that unites software development (Dev) and IT operations (Ops) teams to build, test, and release software faster and more reliably. Traditionally, developers wrote code and handed it over to a separate operations team for deployment, which often caused delays, errors, and communication gaps.

Under a DevOps model:

- Teams work together across the entire application lifecycle.
- Manual, slow processes are replaced with automated workflows.
- Responsibility is shared, meaning developers care about how code runs in production and operations are involved from the start.



The DevOps Lifecycle is a continuous, automated loop that connects software development and IT operations to deliver updates faster and more reliably. The DevOps lifecycle contains CI/CD (Continuous Integration and Continuous Delivery/Deployment) as its core automated backbone.

DevOps Key Practices

- **CI/CD (Continuous Integration and Continuous Delivery):** Automatically building, testing, and preparing code changes for release.
- **Automation:** Using tools to handle repetitive tasks like testing and infrastructure setup.
- **Infrastructure as Code (IaC):** Managing and provisioning computer infrastructure through readable definition files rather than manual hardware configuration.
- **Continuous Monitoring:** Tracking system performance and user feedback in real time to catch and fix issues early.
- **DevSecOps:** Integrating security practices into every phase of the development lifecycle rather than leaving security as a final check.

Key Phases of the DevOps Lifecycle

- **Plan:** Define project goals, roadmaps, and user stories using tools like [Jira](#) or Trello.
- **Code:** Write and manage source code changes using version control systems like [Git](#).
- **Build:** Compile code into executable artifacts using tools like Maven or Gradle.
- **Test:** Run automated checks to detect bugs and vulnerabilities using tools like Selenium or JUnit.
- **Release:** Package and approve builds for production readiness.
- **Deploy:** Push code automatically to live environments using platforms like [Docker](#) or Kubernetes.
- **Operate:** Manage infrastructure, scaling, and server health.
- **Monitor:** Track application performance and system logs using tools like Prometheus or Grafana.





Cloud Computing & Dev-Ops



Think Cloud. Act DevOps. Be Unstoppable

Role of Automation

Automation is not just "nice to have"; it is the foundation of the DevOps lifecycle. Without it, the manual overhead of managing complex cloud environments would be impossible. Automation extends to provisioning servers (Infrastructure as Code), testing (Automated Testing Frameworks), and deployment (Pipeline Orchestration). It removes the human element from repetitive tasks, ensuring that processes are executed consistently every single time.

Pipeline

A pipeline is a set of connected steps or processes that move items, data, or instructions from a starting point to an ending point automatically and in order.

With respect to Computing and Software, DevOps / CI/CD Pipeline is describing pipeline as an automated workflow that takes software code from development, runs tests, and deploys it to production with little manual effort. A DevOps pipeline is a set of automated steps and tools that moves code from a developer's computer to live production safely and quickly.

Key Stages of a Pipeline

- **Source/Commit:** Developers write code and save it to a shared system like GitHub.
- **Build:** The system compiles the code and packages it into a ready-to-run file.
- **Test:** Automated programs check the code for bugs, errors, and security issues.
- **Deploy:** The validated code is automatically sent to testing, staging, or live user environments.
- **Monitor:** Tools track the app's performance in real time to catch new problems fast.

Main Benefits

- **Less manual work:** Automation replaces slow, error-prone human handoffs.
- **Faster releases:** Teams can ship updates and fixes multiple times a day.
- **Higher quality:** Bugs are caught early before users ever see them.

CI/CD

It is an automated software development workflow that builds, tests, and delivers code updates without manual delays. It bridges the gap between software developers who write code and IT operations teams who manage servers.

- **Continuous Integration (CI):** The practice of automating the building and testing of code every time a developer commits changes.
- **Continuous Delivery (CD):** The practice of ensuring that the code is *always* in a deployable state, though the actual deployment to production might be a manual trigger.
- **Continuous Deployment:** The practice where every change that passes the pipeline is *automatically* deployed to production without manual intervention.

Implementing a **CI/CD pipeline** shifts software delivery from error-prone manual deployments to a predictable, fully automated workflow. It bridges the gap between writing code and shipping a live application.





Cloud Computing & Dev-Ops



Think Cloud. Act DevOps. Be Unstoppable

Blueprint of a Modern CI/CD Pipeline

A robust pipeline relies on a chain of distinct validation stages. If a stage fails, the pipeline halts immediately, giving engineering teams rapid feedback.

[Code Commit] —► [Build] —► [Test & Scan] —► [Deploy to Staging] —► [Production Rollout]

Source & Linting: Triggered automatically whenever a developer pushes code to a repository like **GitHub**. Pre-commit hooks check syntax formatting (linting) locally before the code even leaves the developer's laptop.

Build Automation: The pipeline compiles source code and packages it into an immutable artifact, usually a **Docker container image**.

Automated Testing: The pipeline subjects the artifact to immediate checks, adhering to the **testing pyramid**:

- **Unit Tests:** Run in seconds to evaluate micro-logic.
- **Integration/End-to-End Tests:** Spin up ephemeral testing environments to check how database connections and external systems talk to each other.

Security Gates (DevSecOps): Automatically scans packages for vulnerabilities using **SonarQube** or **Trivy** before pushing artifacts to a registry.

Environment Promotion: Code moves sequentially from development to **Staging** (an environment mirroring production) for final checks before passing structural approvals to enter **Production**.

Industry-Standard Tools

Modern workflows define pipelines as declarative code (usually written in YAML), enabling configuration file tracking via standard Git versioning.

Tooling Platform	Best Suited For	Deployment Control Style
GitHub Actions	Native GitHub ecosystems; easy setup via repository workflows.	Event-driven scripts.
GitLab CI/CD	Enterprise-grade, unified single-application platform.	Native integrated container registries.
Argo CD	Kubernetes orchestration environments.	GitOps: Automated continuous reconciliation loops.
Jenkins	Complex, highly customized legacy infrastructures.	Self-hosted engine running plugins.

Production Deployment Frameworks

To implement the final "CD" phase safely without risking downtime, teams leverage two advanced traffic-routing models:

- **Blue-Green Deployment:** Maintains **two identical server environments** simultaneously. "Blue" serves live production traffic while the new code is deployed to "Green". Once verified, the load balancer shifts 100% of the traffic to Green. If something breaks, rolling back is as simple as flipping the router switch back to Blue.





Cloud Computing & Dev-Ops



Think Cloud. Act DevOps. Be Unstoppable

- **Canary Deployment:** Gradually rolls out changes to a **small segment of users** first (e.g., 5% of web traffic). The pipeline monitors errors and metrics on this group; if no bugs occur, it slowly scales the release up to 100% of the production cluster.

Implementation Best Practices

- **Build Once, Promote Everywhere:** Never rebuild container images for different environments. Compile your Docker image once in the build stage, tag it, and pass that precise artifact through Staging and into Production.
- **Leverage Caching:** Drastically reduce execution times by caching package manager dependencies (e.g., node_modules, Maven caches) across runs.
- **Zero Hard-Coded Secrets:** Inject database passwords and cloud API tokens securely at runtime using credential vaults like HashiCorp Vault or repository secrets variables.

Pipeline? Which Language I will write?

DevOps pipelines are primarily created and configured using **YAML** or **Groovy**, depending on the specific Continuous Integration and Continuous Delivery (CI/CD) tool being used.

Because modern DevOps practices rely heavily on **Pipeline-as-Code**, configurations are defined in text files rather than manual UI clicks. Here is a breakdown of the primary languages used across the industry:

1. Configuration & Data Serialization Languages (The Standard)

Most modern CI/CD platforms use data serialization languages to define pipelines declaratively.

- **YAML (.yaml / .yml):** This is the industry standard for defining modern declarative pipelines. You use it to define stages, steps, environments, and commands.
 - *Where it's used:* GitHub Actions (main.yml), GitLab CI/CD (.gitlab-ci.yml), Azure Pipelines (azure-pipelines.yml), AWS CodePipeline, and Bitbucket Pipelines.

2. Dedicated Programming & Scripting Languages

When a pipeline requires complex logic, conditional workflows, or heavy automation scripting, engineers rely on programming languages.

- **Groovy:** A Java-based scripting language that is mandatory for defining complex workflows and advanced pipeline-as-code structures in **Jenkins** (via a Jenkinsfile).
- **Bash / Shell Scripting:** Used *inside* the pipeline stages to execute command-line operations, handle file management, and trigger software builds on Linux-based runners.
- **PowerShell:** Used inside pipelines for task automation and system administration in Windows environments.
- **Python:** Often used as "glue code" inside a pipeline to orchestrate complex tasks, interact with cloud provider APIs, and manage advanced automation logic that goes beyond simple shell scripts.

Pipeline Tools and Their Languages

While the pipeline configuration itself is written in the languages below, the pipeline can build, test, and deploy code written in any programming language, such as Java, Python, Go, or C#.





Cloud Computing & Dev-Ops



Think Cloud. Act DevOps. Be Unstoppable

DevOps Tool	Primary Language Used for Pipeline Creation
GitHub Actions	YAML
GitLab CI/CD	YAML
Azure Pipelines	YAML (or Classic UI editor)
Jenkins	Groovy (via Jenkinsfile)
CircleCI	YAML
Argo CD / Tekton	YAML (Kubernetes-native)

What is YAML?

YAML is a lightweight, human-readable data serialization language used primarily for configuration files and data exchange. The name stands for **YAML Ain't Markup Language** (a recursive acronym, originally *Yet Another Markup Language*).

It is like XML and JSON but has easy and mini syntax. Previously we used XML, then JSON came, and now YAML. It is quite easy, and we can represent the complex mapping in a straightforward way. Due to this nowadays YAML is used in configuration settings.

YAML:

- **Y**: YAML
- **A**: Ain't
- **M**: Markup
- **L**: Language



xml	json	yaml
<pre><?xml version="1.0" encoding="UTF-8"?> <Customers> <Customer> <CustomerId>101</CustomerId> <CustomerName>Alice Smith</CustomerName> <CustomerEmail>alice.smith@example.com</CustomerEmail> <Status>Active</Status> </Customer> <Customer> <CustomerId>102</CustomerId> <CustomerName>Bob Jones</CustomerName> <CustomerEmail>bob.jones@example.com</CustomerEmail> <Status>Inactive</Status> </Customer> </Customers></pre>	<pre>{ [{ "CustomerId": 101, "CustomerName": "Alice Smith", "CustomerEmail": "alice.smith@example.com", "Status": "Active" }, { "CustomerId": 102, "CustomerName": "Bob Jones", "CustomerEmail": "bob.jones@example.com", "Status": "Inactive" }] }</pre>	<pre>--- customers: - CustomerId: 101 CustomerName: "Alice Smith" CustomerEmail: "alice.smith@example.com" Status: "Active" - CustomerId: 102 CustomerName: "Bob Jones" CustomerEmail: "bob.jones@example.com" Status: "Inactive"</pre>

YAML is widely used for:

- **Configuration files** in DevOps, CI/CD, Kubernetes, and Docker.
- **Data exchange** between different programming languages.
- **Infrastructure as Code (IaC)** tools like Ansible and Terraform.

Key Features of YAML

- **Human-Readable:** Designed to look clean and minimalistic, resembling natural language.
- **Indentation-Based:** Uses spaces for structure and hierarchy rather than curly braces {} or brackets [] (like Python). Tab characters are strictly forbidden.
- **Key-Value Pairs:** Organizes core data using simple key: value mappings.
- **Superset of JSON:** Any valid JSON file is also a valid YAML file.
- **Comments:** Supports comments using the hash/pound symbol (#).
- **File Extensions:** Saved with .yaml or .yml extensions.



Cloud Computing & Dev-Ops



Think Cloud. Act DevOps. Be Unstoppable

A YAML document represents a computer program's native data structure in a human readable text form. A node in a YAML document can have three basic data types:

- **Scalar:** Atomic data types like strings, numbers, booleans and null
- **Sequence:** A list of nodes
- **Mapping:** A map of nodes to nodes. Also known as Hashes, Hash Maps, Dictionaries or Objects. Unlike in many programming languages, a key can be more than just a string. It can be a sequence or mapping itself.

On top of that, YAML allows to serialize all other data types and classes:

- **Alias and Anchor:** For serializing References / Pointers, including circular references.
- **Tag:** With Tags it's possible to define custom types/classes.

For example, in many languages a Regular Expression is a built-in data type or object. Some languages have only arrays, which are represented by the basic sequence type. But some have tuples, which needs a custom tag. One YAML File (or Stream) can consist of more than one Document.

Documents

A document is a top-level node. A YAML file (stream) usually contains a single document; it may contain 0-n documents. A YAML file or Stream can contain multiple Documents. YAML Libraries typically have functions like load and load_all to read only one or all documents.

Key Characteristics of a YAML Document

- **Top-Level Node:** It represents a complete root data structure, which can be a scalar, a sequence (list), or a mapping (dictionary).
- **Multiple Documents:** A single YAML file (or stream) can contain zero, one, or multiple separate documents.
- **Separation (---):** Individual documents in a multi-document file are explicitly separated by three hyphens (---), known as the document start marker.
- **Termination (...):** You can explicitly end a document using three dots (...), though it is optional if a new document or the end of the file follows.
- **Isolated Scopes:** Each document acts as an independent file-like scope. This means duplicate keys are allowed across different documents in the same file, but not within the same single document.

YAML Creation

Steps to Create a YAML File:

- Open a plain text editor like Visual Studio Code, Notepad++, or Sublime Text.
- Add key-value pairs using a colon and a space after the key (e.g., name: value).
- Use spaces for indentation to show nested structures or child elements. Never use tab characters.
- Start list items with a hyphen and a space (- item).
- Save your document with either a .yaml or .yml file extension.





Example:

Here is a step-by-step introduction to YAML syntax using a customer and order management example.

Step 1: Key-Value Pairs and Scalars

Every YAML document is built out of key-value pairs. YAML does not use quotation marks for standard text strings or numbers (collectively called scalars).

```
id: CUST-89021
name: Evelyn Harper
email: evelyn.h@example.com
is_premium: true
```

Syntax & Standards

- Case Sensitivity: YAML is case-sensitive (id and ID are different).
- Colons: A colon: must always be followed by a space before the value. id:CUST-89021 is invalid.
- Data Types: YAML automatically detects data types like strings (text), integers, floats, and booleans (true/false).

Step 2: Dictionaries (Mappings)

To group related data together, you nest key-value pairs. This creates a dictionary or object.

```
id: CUST-89021
name: Evelyn Harper
address:
  street: 742 Evergreen Terrace
  city: Springfield
  postal_code: 62704
```

Syntax & Standards

- Indentation: YAML uses spaces for indentation, not tabs. Tab characters will throw error.
- Structure: Nested elements must be indented consistently (usually 2 or 4 spaces) beneath their parent key.

Step 3: Lists (Sequences)

To represent multiple items—like a history of orders—you use a list. List items start with a hyphen - followed by a space.

```
id: CUST-89021
name: Evelyn Harper
recent_orders:
  - ORD-1001
  - ORD-1002
  - ORD-1005
```



Syntax & Standards

- Hyphen Space: The hyphen - must have a space after it.
- Order Preservation: Lists are ordered sequences. The position of items in a list matters, unlike keys in a dictionary.

Step 4: Putting It Together (List of Mappings)

let's look at a complex structure: a list where each item is its own dictionary containing order details.

```
id: CUST-89021
name: Evelyn Harper
orders:
  - order_id: ORD-1001
    date: 2026-09-20
    total: 128.50
    items:
      - product: Ergonomic Keyboard
        quantity: 1
  - order_id: ORD-1002
    date: 2026-09-23
    total: 45.00
    items:
      - product: Wireless Mouse
        quantity: 1
      - product: Mouse Pad
        quantity: 2
```

Syntax & Standards

- Block Style: The structure above uses standard Block Style, relying clean lines and indentation to define structure.

Step 5: Advanced Features (Multi-line Text & Flow Style)

Sometimes customer records need multi-line notes, or you want to save vertical space using Flow Style (JSON-like syntax).

```
id: CUST-89021
name: Evelyn Harper
# Notes use "|" to keep line breaks exactly as written
delivery_notes: |
  Leave package behind the large flower pot.
  Do not ring the bell (baby sleeping).
# Flow style uses curly braces for maps and square brackets for
lists
preferred_categories: [ "Electronics", "Home Office" ]
```



Cloud Computing & Dev-Ops



Think Cloud. Act DevOps. Be Unstoppable

Syntax & Standards

- Comments: Any text following a # is a comment and is ignored by parsers.
- The Literal Block Indicator (|): Retains all newlines and formatting exactly as typed into the paragraph.
- The Folded Block Indicator (>): (Alternative) Replaces single newlines with spaces, turning a multi-line paragraph into one continuous line.
- Flow Style: Allows you to write lists using [...] and dictionaries using {...} on a single line when compact spacing is preferred.

Complete Valid YAML Document

Here is the final consolidated document starting with ---, which explicitly marks the beginning of a YAML file.

```
---
customer:
  id: CUST-89021
  name: Evelyn Harper
  email: evelyn.h@example.com
  is_premium: true
  address:
    street: 742 Evergreen Terrace
    city: Springfield
    postal_code: 62704
  delivery_notes: |
    Leave package behind the large flower pot.
    Do not ring the bell.
  preferred_categories: [ "Electronics", "Home Office" ]
  orders:
    - order_id: ORD-1001
      date: 2026-09-20
      total: 128.50
      items:
        - product: Ergonomic Keyboard
          quantity: 1
    - order_id: ORD-1002
      date: 2026-09-23
      total: 45.00
      items:
        - product: Wireless Mouse
          quantity: 1
        - product: Mouse Pad
          quantity: 2
...
```

(Note: The ... at the very end is optional and signals the official end of the document).





Cloud Computing & Dev-Ops



Think Cloud. Act DevOps. Be Unstoppable

GitHub Actions



What is GitHub?

GitHub is a web-based platform that helps developers store, manage, and collaborate on code using Git version control. It allows multiple developers to work on a project simultaneously, also it helps to track changes, and maintain code history.

What are GitHub Actions?

GitHub Actions is an automation tool that allows developers to create CI/CD (Continuous Integration/Continuous Deployment) pipelines. It helps automate various development tasks like testing, building, and deploying applications.



GitHub Actions makes it easy to automate all your software workflows, now with world-class CI/CD. Build, test, and deploy your code right from GitHub. Make code reviews, branch management, and issue triaging work the way you want.

GitHub Actions are the primary mechanism for automation within GitHub. They can be used for a wide variety of purposes, but one of the most common is to set up Continuous Integration/ Continuous Delivery.

Automate, customize, and execute your software development workflows right in your repository with GitHub Actions. You can discover, create, and share actions to perform any job you'd like, including CI/CD, and combine actions in a completely customized workflow.



Core Concepts

- **Actions** are individual tasks that you can combine to create custom workflows. Think of them as reusable units of code that perform specific functions.
- **Workflows** are automated processes made up of one or more actions. They run when triggered by events in your repository.

What is a GitHub Workflow?

A GitHub Workflow is a set of automated tasks defined in a YAML file. It runs when triggered by specific events, such as pushing code to a repository, creating a pull request, or setting up a schedule. Workflows help in automating repetitive tasks like testing, code analysis, and deployment.





Cloud Computing & Dev-Ops



Think Cloud. Act DevOps. Be Unstoppable

What is a GitHub Runner?

A GitHub Runner is a virtual machine that executes jobs specified in a GitHub Actions workflow. GitHub provides both hosted runners (free-tier available) and self-hosted runners (custom machines for private execution).

Common use cases

GitHub Actions excels at automating various development tasks:

CI/CD pipelines

- **Continuous Integration:** Automatically build and test code on every push or pull request.
- **Continuous Deployment:** Deploy applications to staging or production environments.
- **Multi-environment deployments:** Deploy to different environments based on branch rules.

Code quality and security

- **Automated testing:** Run unit tests, integration tests, and end-to-end tests.
- **Code analysis:** Perform static code analysis and security scanning.
- **Dependency management:** Automatically update dependencies and check for vulnerabilities.

Project management automation

- **Issue triage:** Automatically label and assign new issues.
- **Pull request management:** Auto-merge dependabot updates or require reviews.
- **Release automation:** Create releases and generate changelogs automatically.

How Actions work

Actions are defined using YAML files stored in the `.github/workflows` directory of your repository. These workflows:

1. **Trigger** on specific events (push, pull request, schedule, etc.).
2. **Run** on virtual machines called "runners" (GitHub-hosted or self-hosted).
3. **Execute** a series of steps that can include pre-built actions or custom scripts.

Action ecosystem

The GitHub Marketplace provides thousands of pre-built actions created by GitHub and the community. Popular categories include:

- **Deploy actions:** Deploy to Azure and other platforms.
- **Testing actions:** Run tests with various frameworks and tools.
- **Security actions:** Scan for vulnerabilities and security issues.
- **Notification actions:** Send messages to Teams, or email.

You can also create custom actions using Docker containers, JavaScript, or composite actions that combine multiple steps.





Cloud Computing & Dev-Ops



Think Cloud. Act DevOps. Be Unstoppable

Creating First GitHub Workflow

You can create workflows that run tests whenever you push a change to your repository, or that deploy merged pull requests to production. Before start make sure you have at least a basic knowledge of how to use GitHub, working with repositories, branches etc.

Create a New GitHub Repository.

In your repository on GitHub, create a workflow file called **github-actions-demo.yml** in the **.github/workflows** directory. To do this you can follow options:

- If the **.github/workflows** directory already exists, navigate to that directory on GitHub, click Add file, then click Create new file, and name the file **github-actions-demo.yml**.
- If your repository doesn't have a **.github/workflows** directory, go to the main page of the repository on GitHub, click Add file, then click Create new file, and name the file **.github/workflows/github-actions-demo.yml**. This creates the **.github** and **workflows** directories and the **github-actions-demo.yml** file in a single step.

For GitHub to discover any **GitHub Actions Workflows** in your repository, you must save the workflow files in a directory called **.github/workflows**. You can give the workflow file any name you like, but you must use **.yml** or **.yaml** as the file name extension.

Code in following YAML contents into the **github-actions-demo.yml** file:

```
name: GitHub Actions Demo
run-name: ${{ github.actor }} is testing out GitHub Actions 🚀
on: [push]
jobs:
  Explore-GitHub-Actions:
    runs-on: ubuntu-latest
    steps:
      - run: echo "🎉 The job was automatically triggered by a ${{
github.event_name }} event."
      - run: echo "🐙 This job is now running on a ${{ runner.os }} server
hosted by GitHub!"
      - run: echo "🔍 The name of your branch is ${{ github.ref }} and
your repository is ${{ github.repository }}."
      - name: Check out repository code
        uses: actions/checkout@v6
      - run: echo "💡 The ${{ github.repository }} repository has been
cloned to the runner."
      - run: echo "📄 The workflow is now ready to test your code on the
runner."
      - name: List files in the repository
        run: |
          ls ${{ github.workspace }}
      - run: echo "🍏 This job's status is ${{ job.status }}."
```

At this stage you don't need to understand the details of this workflow. For now, you can just copy and paste the contents into the file. Further we will learn about the syntax of workflow files, GitHub Actions contexts such as `${{ github.actor }}` and `${{ github.event_name }}`.

Using GitHub GUI to create workflow file





Cloud Computing & Dev-Ops

Think Cloud. Act DevOps. Be Unstoppable



Create a new repository

Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).
Required fields are marked with an asterisk (*).

1 General

Owner * RanjanBhatnagar / Repository name * GitActionsDemo
Great repository names are short and memorable. How about [shiny-goggles](#)?
Description Learning Git Actions Workflows for CI/CD
41 / 350 characters

2 Configuration

Choose visibility * Public
Choose who can see and commit to this repository

Add README
READMEs can be used as longer descriptions. [About READMEs](#) Off

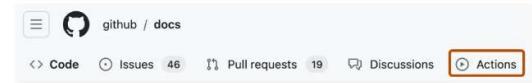
Add .gitignore
.gitignore tells git which files not to track. [About ignoring files](#) No .gitignore

Add license
Licenses explain how others can use your code. [About licenses](#) No license

Create repository

You can use this repository as normal repo only.

On the navigation bar you have Actions button.



You will get Get started with GitHub Actions option. It allows: Build, test, and deploy your code. Make code reviews, branch management, and issue triaging work the way you want. Select a workflow to get started. Click on link:

Skip this and [set up a workflow yourself](#) →

You will get blank yml file window, type in and

proceed with.

GitActionsDemo / .github / workflows / main.yml in main

Edit Preview

1 Enter file contents here

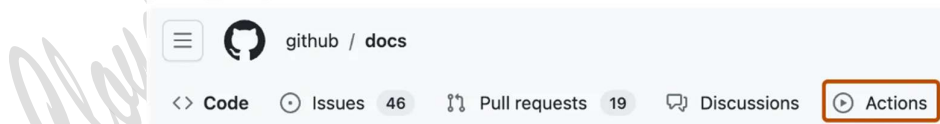
Create workflow as in previous sections.

Click Commit changes.

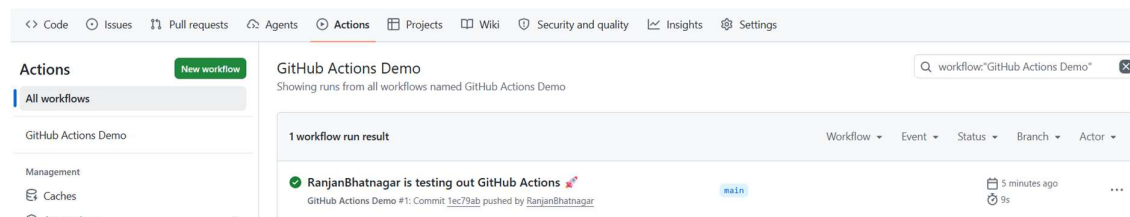
Committing the workflow file to a branch in your repository triggers the push event and runs your workflow.

Viewing Workflow Results

On GitHub, navigate to the main page of the repository. Under your repository name, click Actions.



In the left sidebar, click the workflow you want to display, in this example "GitHub Actions Demo."





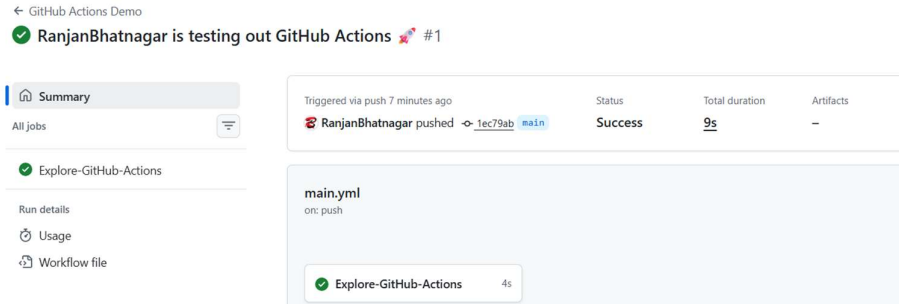
Cloud Computing & Dev-Ops



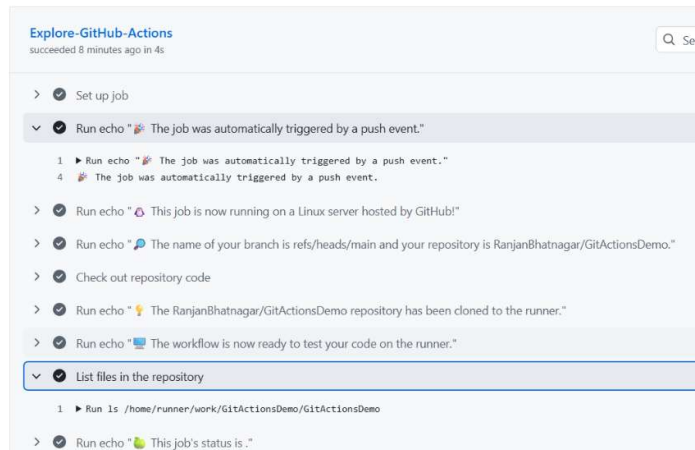
Think Cloud. Act DevOps. Be Unstoppable

From the list of workflow runs, click the name of the run you want to see, in this example "USERNAME is testing out GitHub Actions."

In the left sidebar of the workflow run page, under Jobs, click the Explore-GitHub-Actions job.



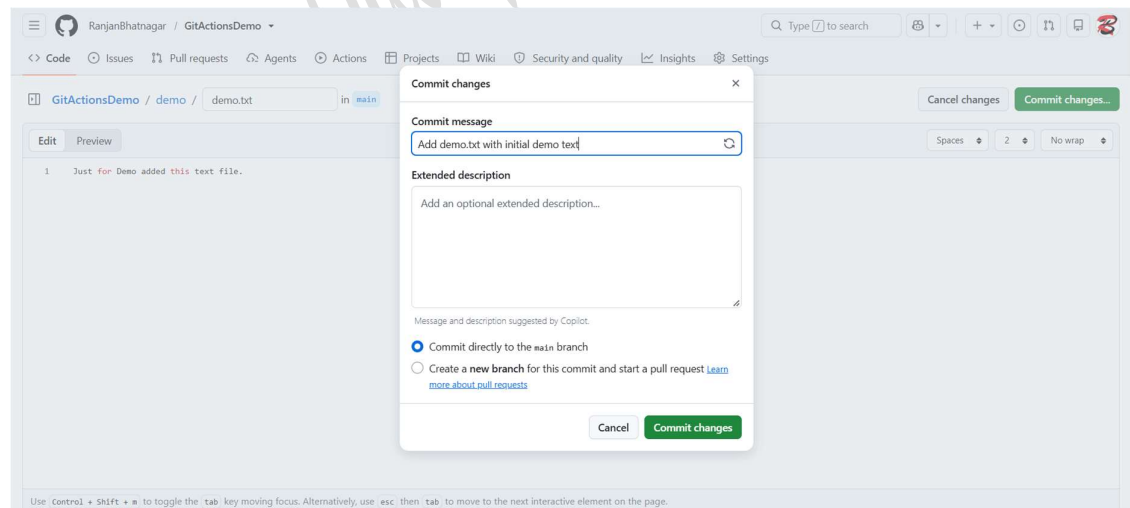
The log shows you how each of the steps was processed. Expand any of the steps to view its details.



The example workflow you just added is triggered each time code is pushed to the branch, and shows you how GitHub Actions can work with the contents of your repository.

Let's try little more in same workflow to see how automation works or you can see CI works. On GitHub, navigate to the main page of the repository. Try adding a new text file in a folder demofiles\demo.txt. add some text into file and use commit

changes as usual. Basically, we want to push any new element to repository.



Now immediately go to Actions and see automatically CI triggered and workflow is running. It is queued as below:





Cloud Computing & Dev-Ops



Think Cloud. Act DevOps. Be Unstoppable

2 workflow runs

- RanjanBhatnagar is testing out GitHub Actions
GitHub Actions Demo #2: Commit [15327e9](#) pushed by [RanjanBhatnagar](#)
- RanjanBhatnagar is testing out GitHub Actions
GitHub Actions Demo #1: Commit [1ec79ab](#) pushed by [RanjanBhatnagar](#)

After sometime/completion you can see a green tick.

2 workflow runs

Workflow ▾ Event ▾ Status ▾ Branch ▾ Actor ▾

RanjanBhatnagar is testing out GitHub Actions GitHub Actions Demo #2: Commit 15327e9 pushed by RanjanBhatnagar	main	now 11s	...
RanjanBhatnagar is testing out GitHub Actions GitHub Actions Demo #1: Commit 1ec79ab pushed by RanjanBhatnagar	main	17 minutes ago 9s	...

Next, go to `.github/workflows/main.yml` and make changes for example

In document - name: List files in the repository is running a command

run: |

ls \${{ github.workspace }}

change it to ls with recursive option so it will list all files in folders and subfolders also.

ls -R \${{ github.workspace }}

Commit Changes. You can view 3rd workflow queued:

3 workflow runs

Workflow ▾ Event ▾ Status ▾ Branch ▾ Actor ▾

RanjanBhatnagar is testing out GitHub Actions GitHub Actions Demo #3: Commit 053426c pushed by RanjanBhatnagar	main	now Queued	...
---	------	---------------	-----

Once completed open in by clicking and then click on action and expand List files in repository section and you can observe listing all files in Repository.

> Run echo "🖥️ The workflow is now ready to test your code on the runner."

> List files in the repository

1 ▶ Run **ls -R /home/runner/work/GitActionsDemo/GitActionsDemo**

4 /home/runner/work/GitActionsDemo/GitActionsDemo:

5 demo

6

7 /home/runner/work/GitActionsDemo/GitActionsDemo/demo:

8 demo.txt

> Run echo "🍏 This job's status is."



Cloud Computing & Dev-Ops



Think Cloud. Act DevOps. Be Unstoppable

Understanding GitHub Actions

GitHub Actions is a continuous integration and continuous delivery (CI/CD) platform that allows you to automate your build, test, and deployment pipeline. You can create workflows that build and test every pull request to your repository, or deploy merged pull requests to production.

GitHub Actions goes beyond just DevOps and lets you run workflows when other events happen in your repository. For example, you can run a workflow to automatically add the appropriate labels whenever someone creates a new issue in your repository.

GitHub provides Linux, Windows, and macOS virtual machines to run your workflows, or you can host your own self-hosted runners in your own data center or cloud infrastructure.

The components of GitHub Actions

You can configure a GitHub Actions **Workflow** to be triggered when an **Event** occurs in your repository, such as a pull request being opened or an issue being created. Your workflow contains one or more **Jobs** which can run in sequential order or in parallel. Each job will run inside its own virtual machine **Runner**, or inside a container, and has one or more **Steps** that either run a script that you define or run an **Action**, which is a reusable extension that can simplify your workflow.



Workflows

A **workflow** is a configurable automated process that will run one or more jobs. Workflows are defined by a YAML file checked in to your repository and will run when triggered by an event in your repository, or they can be triggered manually, or at a defined schedule. Workflows are defined in the **.github/workflows** directory in a repository. A repository can have multiple workflows, each of which can perform a different set of tasks such as:

- Building and testing pull requests
- Deploying your application every time a release is created
- Adding a label whenever a new issue is opened

You can reference a workflow within another workflow.





Cloud Computing & Dev-Ops



Think Cloud. Act DevOps. Be Unstoppable

Events

An **event** is a specific activity in a repository that triggers a **workflow** run. For example, an activity can originate from GitHub when someone creates a pull request, opens an issue, or pushes a commit to a repository. You can also trigger a workflow to run on a schedule, by posting to a REST API, or manually.

Jobs

A **job** is a set of **steps** in a workflow that is executed on the same **runner**. Each step is either a shell script that will be executed, or an **action** that will be run. Steps are executed in order and are dependent on each other. Since each step is executed on the same runner, you can share data from one step to another. For example, you can have a step that builds your application followed by a step that tests the application that was built.

Actions

An action is a pre-defined, reusable set of jobs or code that performs specific tasks within a workflow, reducing the amount of repetitive code you write in your workflow files. Actions can perform tasks such as:

- Pulling your Git repository from GitHub
- Setting up the correct toolchain for your build environment
- Setting up authentication to your cloud provider

You can write your own actions, or you can find actions to use in your workflows in the GitHub Marketplace.

Runners

A runner is a server that runs your workflows when they're triggered. Each runner can run a single job at a time. GitHub provides Ubuntu Linux, Microsoft Windows, and macOS runners to run your workflows. Each workflow run executes in a fresh, newly-provisioned virtual machine.

GitHub also offers larger runners, which are available in larger configurations. If you need a different operating system or require a specific hardware configuration, you can host your own runners.

🔒🔒 Commit to Learning Today, Deploy Success Tomorrow. 🔒🔒

